Milestone Report: Vectorized Loop Shape Optimization in MLIR

Muhammad Mazaya (mmazaya) Justin Kim (yutarkk)

November 2025

1 Major Changes

We are thinking to add more shapes as a consideration, primarily: scalar remainder unrolling, mask remainder or fully mask, (optionally) loop versioning for unknown size of vector use. We will also just focus on 1 dialect: `linalg.matmul`. As time permits, we can also address other dialects such as `linalg.dot`, `linalg.conv_2d`, etc. We have also narrowed down the vector instructions that we want to test to be AVX, AVX2, AVX-512, SVE, and SME.

2 What You Have Accomplished So Far

We have implemented the initial pass to turn a `linalg.matmul` into a vectorized loop with scalar remainder, the result for AVX is viewable in this URL. We have also setup AWS instances with support for AVX, AVX2, AVX-512, SVE, and SME instructions, all of which are running Ubuntu 24.04.

3 Meeting Your Milestone

Yes, we meet our milestone. We have a functional Linalg→Vector prototype for a static shape, producing valid vector dialect IR suitable as a baseline for more strategies.

4 Surprises

Lowering vector dialect operations into actual assembly instructions (with AVX) is more complex than anticipated: the pipeline must pass through multiple dialects (vector → LLVM dialect → LLVM IR → assembly), and we encountered MLIR/LLVM version compatibility issues. Implementing the vector accumulator across the reduction dimension (the k-loop) also turned out to be more challenging than expected. Our initial attempt considered using `vector.reduction` to handle the k-dimension reduction, but `vector.reduction` reduces a vector to a scalar, whereas we need to maintain a vector accumulator that is updated per iteration. We eventually end up with an approach that requires explicitly modeling the k-loop as an `scf.for` with an `iter_args` parameter that carries the accumulator vector from one iteration to the next, updating it with a fused multiply-add inside the loop body, and returning the updated `%acc` in the `scf.yield`.

5 Revised Schedule

- Week 5 (11/20 - 11/27)
  - Muhammad:
    - Unrolled scalar implementation
    - Loop peeling implementation
  - Justin:
    - Remainder masking implementation
    - Full masking implementation
  - Both:
    - Optional: Loop versioning implementation
- Week 6 (11/28–12/4)

- Both:
  - Implement cost model to determine optimal shape strategy
  - Evaluate benchmark and finish final report
  - Poster session preparation

6 Resources Needed

Yes we have all of them, primarily ARM instance for SVE and SME, and x86 instance for AVX, AVX2, and AVX-512.